

DATA266 Lab 1 — Team 6 Report

Team: PairProgramming_Team_06 — Rajesh Paruchuri, Siva Surya Chandran

Repository: https://github.com/Paruchuri-Rajesh/DATA_266-GPULab1

Every number below comes from the committed `metrics_report.csv` / `full_metrics_report.csv` files in each member's folder. Each member's `results.md` gives the full tables and the reasoning behind each choice.

Team ownership statement

Each member independently designed, coded, trained and evaluated their own models for all three tasks.

Rajesh Paruchuri built:

- a 6-layer character-level GPT for TinyStories (Task 1);
- a fastText-bigram baseline, a BiGRU with attention and a from-scratch Transformer encoder for Yelp polarity (Task 2);
- a CycleGAN with 6-block / ngf-48 generators, spectral-norm PatchGAN discriminators, DiffAugment and EMA generators (Task 3).

Siva Surya Chandran built:

- a 6-layer, 8-head, 256-wide character-level GPT trained on a Colab T4 (Task 1);
- a mean-pooled embedding baseline, a multi-width CNN and a BiLSTM with attention for Yelp polarity (Task 2);
- a CycleGAN with 9-block / ngf-64 resize-conv generators, InstanceNorm PatchGAN discriminators, translation DiffAugment and EMA, whose final run was on the lab RTX 4090 (Task 3).

We wrote the comparison tables and joint analyses below together.

Task 1 — GPT-style LLM from scratch (TinyStories)

Metric	Rajesh Paruchuri	Siva Surya Chandran
Architecture	Decoder-only GPT, hand-written LayerNorm / causal MHA / GELU FFN, pre-norm, weight-tied head	Decoder-only GPT, hand-written causal MHA / LayerNorm / GELU FFN, pre-norm, separate (untied) LM head
Layers / heads / width / context	6 / 6 / 192 / 192 chars	6 / 8 / 256 / 128 chars
Data split	Story-disjoint: 100,000 train / 10,000 val non-overlapping windows, seed 266	100,000 train / 10,000 val stories sampled from TinyStories (seed 1337), cut into non-overlapping 128-char windows
Hyperparameters	AdamW (0.9, 0.95), wd 0.1, lr 1e-3, 500-step warm-up + cosine to 1e-4, batch 64, dropout 0.1, 10 epochs	AdamW (0.9, 0.95), wd 0.01, lr 3e-4, 500-step warm-up + cosine to 3e-5, batch 64, dropout 0.1, 10 epochs, fp16
Parameters	2,719,488	4,827,648
Train CE (eval mode) / Val CE	0.6920 / 0.7111	0.6303 / 0.6385
Perplexity / bits per char	2.036 / 1.026	1.894 / 0.921
Generalisation gap (val – train)	+0.019	+0.008
Top-1 next-char accuracy	0.7746	0.7955
Distinct-1/2/3, sampled	0.387 / 0.801 / 0.935 (T = 0.7)	0.252 / 0.676 / 0.886 (T = 0.8)
Repeated 4-gram rate (greedy / sampled)	0.396 / 0.005 (T = 0.7)	0.267 / 0.006 (T = 0.8)
Grad norm mean / max · NaNs · loss spikes	0.386 / 8.75 · 0 · 0	0.343 / 7.77 · 0 · 0 (41 fp16 overflow steps skipped by the scaler)
Train / generation tokens/s	18,282 / 100	133,346 / 217
Peak memory · train time	3,308 MB (MPS) · 11,173 s	944 MB (CUDA) · 6,736 s
Hardware	Apple M4 MacBook Air, MPS	NVIDIA Tesla T4 (Google Colab), fp16

Evidence:

	Rajesh	Siva
Loss curves	task1_llm/rajesh_paruchuri/outputs/loss_curves_full.png	task1_llm/sivasurya_chandran/checkpoints/loss_curves.png

	Rajesh	Siva
Samples	outputs/generated_samples_full.txt	outputs/samples.txt
Checkpoints	checkpoints/best.pt	checkpoints/gpt_final.pt
Raw logs	reproducibility/raw_logs/<member>/task1_llm/	reproducibility/raw_logs/<member>/task1_llm/

Joint analysis.

Rajesh's points:

- The story-disjoint split makes validation a true out-of-story test.
- Training was stable (no spikes; gradient norm about 0.35 after warm-up).
- Greedy decoding loops (repeated 4-gram rate 0.40), while T = 0.7 does not.
- Coherence breaks down beyond about 2 sentences (speaker and entity drift).
- Next steps: KV cache for generation, BPE tokens, longer training.

Siva's points:

- The larger model (4.8M vs 2.7M parameters) reaches a lower validation CE (0.639 vs 0.711). The gap between train and val is tiny (0.008), so it is not overfitting and could take more capacity.
- The two numbers are not a strict like-for-like comparison. Rajesh's story-disjoint split is harder, and his longer 192-character context should help with exactly the coherence problems we both see.
- fp16 on the T4 trained about 7× faster per second than fp32 on the M4. The GradScaler skipped only 41 of 109,640 steps.

Together:

- Both models learn spelling and local grammar well (77–80% next-character accuracy).
- Both fail in the same ways: greedy decoding loops, characters change names or pronouns once they leave the context window, and some sampled events make no sense.
- **Strengths:** stable training from scratch with warm-up + cosine, no NaNs, small generalisation gaps.
- **Weaknesses:** short memory (128–192 characters) and no notion of who is who across sentences.
- **Next steps** we agree on: a longer context or BPE tokens, top-k / nucleus sampling or a repetition penalty, and a larger model, since neither of us overfits.

Task 2 — Yelp polarity sentiment classification

Metric	Rajesh: fastText bigram (baseline)	Rajesh: BiGRU + attention	Rajesh: Transformer	Siva: bag-of-embeddings (baseline)	Siva: CNN (exp 1)	Siva: BiLSTM + attention (exp 2)
Architecture	uni + hashed-bigram embedding bag → linear	BiGRU 128/dir + additive attention	2-layer encoder, 4 heads, [CLS]	mean-pooled unigram embeddings → MLP (hidden 128)	1-D CNN, kernels 3/4/5 × 128 filters, max-over-time pooling	BiLSTM 128/dir (packed) + additive attention
Key hyperparameters	emb 64, lr 2e-3, 4 ep	emb 128, lr 1e-3, 3 ep	emb 128, lr 5e-4, 3 ep	emb 128, lr 2e-3, dropout 0.3, ≤ 5 ep	emb 128, lr 1e-3, dropout 0.5, ≤ 5 ep	emb 128, lr 1e-3, dropout 0.3, ≤ 5 ep
Accuracy	0.9515	0.9523	0.9323	0.9332	0.9502	0.9581
Precision / recall / F1 (macro)	0.9515 / 0.9515 / 0.9515	0.9523 / 0.9523 / 0.9523	0.9323 / 0.9323 / 0.9323	0.9332 / 0.9332 / 0.9332	0.9502 / 0.9502 / 0.9502	0.9581 / 0.9581 / 0.9581
F1 micro / weighted	0.9515 / 0.9515	0.9523 / 0.9523	0.9323 / 0.9323	0.9332 / 0.9332	0.9502 / 0.9502	0.9581 / 0.9581
ROC-AUC / PR-AUC	0.9874 / 0.9872	0.9903 / 0.9906	0.9830 / 0.9837	0.9812 / 0.9816	0.9883 / 0.9882	0.9921 / 0.9923
MCC	0.9030	0.9046	0.8645	0.8665	0.9004	0.9161
Brier / ECE	0.0382 / 0.0145	0.0363 / 0.0147	0.0494 / 0.0079	0.0503 / 0.0082	0.0384 / 0.0105	0.0323 / 0.0124
95% CI accuracy	[0.9496, 0.9535]	[0.9502, 0.9545]	[0.9298, 0.9346]	[0.9309, 0.9359]	[0.9479, 0.9523]	[0.9560, 0.9601]

Metric	Rajesh: fastText bigram (baseline)	Rajesh: BiGRU + attention	Rajesh: Transformer	Siva: bag-of-embeddings (baseline)	Siva: CNN (exp 1)	Siva: BiLSTM + attention (exp 2)
McNemar vs own baseline	—	exact p = 0.44	exact p = 4×10^{-74}	—	χ^2 p < 10^{-16} (b/c 688 / 1,332)	χ^2 p < 10^{-16} (b/c 542 / 1,485)
Worst slice macro-F1	0.875 (non-English, n = 133)	0.810 (non-English)	0.781 (non-English)	0.924 ("but/however/although")	0.945 ("but/however/although")	0.953 ("but/however/although")
Params · train time	20.26M · 1,195 s	7.20M · 17,403 s	7.26M · 9,084 s	6.42M · 173 s	6.60M · 791 s	6.70M · $\approx 10,600$ s (19,092 s logged, includes $\sim 8,500$ s of laptop sleep)
Peak memory (MPS tensors / driver)	310 / 2,787 MB	127 / 4,032 MB	130 / 10,130 MB	— / 2,810 MB	— / 2,828 MB	— / 4,006 MB
Hardware	Apple M4, MPS	Apple M4, MPS	Apple M4, MPS	Apple M5, MPS	Apple M5, MPS	Apple M5, MPS

Full per-slice tables, confusion matrices and bootstrap CIs for MCC and macro-F1 are in each member's `metrics_report.csv`.

The worst slice differs between members because we defined different slices: Rajesh has a non-English slice; Siva's slices are length, negation, contrast and exclamation-heavy.

Evidence:

- **Rajesh:** `outputs/confusion_roc_pr_calibration_full.png`, `outputs/training_curves_full.png`.
- **Siva:** `outputs/test_curves.png`, `outputs/test_confusion.png`, `outputs/eda/`.
- **Checkpoints:** each member's `checkpoints/*.pt`.
- **Raw logs:** `reproducibility/raw_logs/<member>/task2_sentiment/`.

Joint analysis.

Rajesh's points:

- The BiGRU's lead over the bigram baseline is not significant (McNemar p = 0.44), but their errors differ: averaging the two gives 0.9572 (post hoc).
- The from-scratch Transformer overfits after epoch 1 but is the best calibrated model.
- 6 of 20 reviewed errors come from preprocessing (deleted prices, case and emphasis, the stopword "just", split accents).
- Non-English reviews are the weakest slice.
- Next steps: ordinal 5-star training, a Unicode-aware tokeniser, an ensemble.

Siva's points:

- Each step up in word order helped: bag $\rightarrow$ CNN $\rightarrow$ BiLSTM went from 0.933 to 0.950 to 0.958, and McNemar is significant for both experimental models. The biggest gains were on the negation and contrast slices (contrast error 7.5% $\rightarrow$ 5.4% $\rightarrow$ 4.65%).
- The BiLSTM is the most accurate model but slightly over-confident (ECE 0.012). The baseline is the best calibrated (ECE 0.008).
- 8 of the 20 reviewed errors are contrast clauses, and about 4 are label noise (the star rating disagrees with the text).
- Next steps: head + tail truncation, marking contrast and negation scope, temperature scaling.

Together:

- **Best model.** Siva's BiLSTM + attention is the team's best (0.9581). Its accuracy CI [0.9560, 0.9601] does not overlap Rajesh's best (BiGRU, [0.9502, 0.9545]). Recurrent encoders with attention pooling came out on top in both lineups. The from-scratch Transformer was the weakest experimental model, which fits the small amount of data per parameter without pre-training.
- **Bigrams matter.** Rajesh's bigram baseline (0.9515) is almost two points above Siva's unigram baseline (0.9332), and nearly as good as either recurrent model at a fraction of the cost. Much of what word order buys is in two-word phrases like "not good".
- **Shared weakness.** Contrast reviews ("good food, but ...") are the hardest large slice for every model. Both error reviews also found label noise from the 1–2 vs 3–4 star mapping.
- **Calibration vs accuracy.** The best-calibrated models (Rajesh's Transformer, Siva's baseline) are not the most accurate ones. Temperature scaling on validation would fix the recurrent models cheaply.
- **Cost.** The recurrent models are 15–60 $\times$ slower to train than the bag models on Apple GPUs. For deployment, the bigram baseline or the CNN gives the best accuracy per unit of compute.

Task 3 — CycleGAN photo ↔ Monet

Metric	Rajesh Paruchuri	Siva Surya Chandran
Generators / discriminators	ResNet, 6 blocks, ngf 48, bilinear+conv decoder / spectral-norm 70×70 PatchGAN	ResNet, 9 blocks, ngf 64, nearest-resize+conv decoder / InstanceNorm 70×70 PatchGAN
Training	DiffAugment (colour / translation / cutout), EMA, 128-px crops, 80 epochs × 300 iterations, LSGAN, λ_{cyc} 10, λ_{id} 5	DiffAugment (translation), EMA, 256-px crops, warm start from v2 (50K steps) + 270 epochs × 1,000 iterations, LSGAN, λ_{cyc} 10, λ_{id} 2.5, checkpoint chosen on held-out photos in both directions
Parameters	14,355,336	28,285,832
FID photo→Monet / Monet→photo	105.72 / 148.80 (all images)	83.00 / 89.68 (1,000 held-out photos vs 300 paintings)
KID photo→Monet / Monet→photo	0.0308 / 0.0776	0.0126 / 0.0257
Precision / recall (photo→Monet)	0.247 / 0.523	0.576 / 0.630
Cycle L1 (photo / Monet cycle)	0.0774 / 0.0775	0.1053 / 0.0841
LPIPS input vs translation (photo→Monet)	0.523	0.420
Content cosine (photo→Monet)	0.373 (VGG16 relu4_3)	0.732 (Inception-v3 pool features)
Final losses G-adv / D_A / D_B / cycle / identity	0.645 / 0.214 / 0.216 / 0.358 / 0.373	1.584 / 0.063 / 0.035 / 0.159 / 0.149
Grad norm G / D (mean) · NaNs	44.2 / 5.5 · 0	15.3 / 9.3 · 0
Course-script submission FID / MiFID	149.48 / 0.414	101.91 / 0.408
Kaggle public score	−74.9486	−45.9695 (v3, scored with his earlier scorer; v2 −46.3530; v1 −49.2570). The final model's course-script entry is −51.1585 (submission.csv).
Human audit score · Cohen's kappa	⟨pending: 2 raters⟩	⟨pending: 2 raters⟩
Train time · images/s · peak memory	9,417 s · 5.1 · 409 MB tensors / 1,009 MB pool	24,770 s (final run) · 10.9 steps/s · 19,863 MB (CUDA)
Hardware	Apple M4, MPS	NVIDIA RTX 4090 (lab PC), CUDA, bf16

Kaggle: team PairProgramming_Team_06, public rank **11** (ranked on the team's best submission, Siva's v3).

Evidence:

- **Rajesh:** outputs/loss_curves_full.png, translation_cycle_grid_full.png, src/Part3_Evaluation_Script_run.ipynb.
- **Siva:** checkpoints/loss_curves.png, checkpoints/eval_*_input_fake_rec.png, src/Part3_Evaluation_Script_run.ipynb.
- **Weights:** Siva's checkpoints/{G_AB,G_BA,D_A,D_B}.pt; Rajesh's checkpoints/cyclegan_full.pt (95 MB) is kept out of git; his checkpoints/README.md explains how to recreate it.
- **Raw logs:** reproducibility/raw_logs/<member>/task3_gan/.

Joint analysis.

Rajesh's points:

- Training was stable (no D collapse; D outputs about 0.57 real / 0.43 fake), but under-trained: cycle loss was still falling at epoch 80.
- The cycle constraint verifies: reconstructions are 3.5× closer to the input than unrelated images are. But dark-blob "hidden codes" partly satisfy it.
- Monet→photo is much weaker than photo→Monet.
- Next steps: longer training, full-resolution fine-tuning, noise in the cycle path, a larger Monet→photo generator.

Siva's points:

- The final run took his own v2 model and trained it for 270K more steps on the 4090. The held-out score (course-script formula) improved at all but one of the nine checks, and most of the gain came while the learning rate decayed.
- EMA weights won the first seven of nine checks. By the end the learning rate was near zero and the raw weights won.

- Training was stable: 0 NaN steps in 270,000. The discriminators got stronger as the learning rate fell (G adversarial loss 1.0 $\rightarrow$ 1.5), without hurting the score.
- Visual failures:
 - grey photos are given invented colour;
 - dramatic skies lose contrast;
 - photographer watermarks survive both the translation and the cycle;
 - Monet $\rightarrow$ photo outputs come out too dark.
- Two extra experiments did not help. A UVCGAN-style model collapsed to the identity function. A PatchNCE continuation looked more painterly but scored worse.

Together:

- **Same scorer.** Both submissions were scored with the same course script, so the comparison is direct: 101.91 (Siva) vs 149.48 (Rajesh). Siva's generators are 2.6 $\times$ larger and saw far more training at full 256-px resolution (50K + 270K steps vs 24K steps on 128-px crops). That accounts for most of the gap. Rajesh got a steadier min-max balance from spectral norm with a model half the size.
- **Monet $\rightarrow$ photo is the weak direction for both of us** (Siva 106.9 vs 97.0 photo $\rightarrow$ Monet; Rajesh 169.8 vs 129.2 on the course script). The 300 paintings include subjects (still lifes, interiors) with no photographic counterpart, and inventing photographic detail from a painting is under-determined.
- **Out-of-domain content** is the other shared failure. Both of us saw text, watermarks and frames pass through untranslated, because the identity and cycle losses reward keeping them.
- **Design differences:** both of us used resize-conv upsampling, DiffAugment and EMA. They differ in:
 - capacity (6 blocks / ngf 48 vs 9 blocks / ngf 64);
 - discriminator normalisation (spectral norm vs InstanceNorm);
 - augmentation (colour + translation + cutout vs translation only);
 - identity weight (5 vs 2.5);
 - crop size (128 vs 256).
- **Next steps** we agree on: more training at full resolution, a stronger or reweighted Monet $\rightarrow$ photo direction, noise in the cycle path against hidden codes, and masking watermarks in the photos.

Individual failure and error analyses

Task 1

Rajesh (task1_llm/rajesh_paruchuri/failure_analysis.md):

1. **Repetition loop under greedy decoding.** 4 of 5 greedy samples loop until the 400-character limit.
2. **Lost speaker and entity tracking.** One story opens with "a little boy named Timmy", and three sentences later "*Lily* looked at the tree and said, ...".
3. **Invented words at T = 1.0.** "Tom saw a **bloll** on the floor."; "her mom said it was too **tisty**".

Siva (task1_llm/sivasurya_chandran/failure_analysis.md):

1. **Repetition loop under greedy decoding.** "The sun was shining and the sky was blue. The sun was shining and the sky was blue. ..." (repeated 4-gram rate 0.84 for this sample).
2. **Lost coherence.** "there was a little boy named Timmy ... 'Look, Mommy! I found a shiny rock!' *she* said. ... said *Jack*. ... a big, brown *rocket*". The name, pronoun and object all drift.
3. **Broken grammar and impossible events.** "Suddenly, a big hole in the ground. He picked it up and took it home."

Task 2

Rajesh reviewed 20 BiGRU errors (task2_sentiment/rajesh_paruchuri/failure_analysis.md):

- **Preprocessing.** 6 errors come from his own pipeline (deleted prices, case and emphasis, the stopword "just", split accents).
- **Label noise.** 3 are label noise or a mismatch between text and rating. His most confident false positive, "Wow love the place and everything is very clean and new! Great place to come and relax worth a try!", reads as fully positive.

Siva reviewed 20 BiLSTM errors (task2_sentiment/sivasurya_chandran/failure_analysis.md):

- **Contrast clauses (8).** "Food is good but the portions are small for what you are paying" was predicted positive.

- **Label noise (4).**
- **Truncation at 256 tokens (3).**
- **Past vs present (2).** "This place is so much better since they changed owners... it was terrible... Now its much better" was predicted negative.
- **No sentiment signal (2).** "where is this place?"
- **Negated negatives (1).** "too good to disrespect with a 2 star review".

Task 3

Rajesh (`task3_gan/rajesh_paruchuri/failure_analysis.md`):

- dark or green "hidden code" blobs that carry cycle information;
- bright light sources turned into white blobs with rainbow halos;
- out-of-domain content (frames, text) copied or garbled;
- Monet→photo staying painterly;
- border streaks from training on 128-px crops.

Siva (`task3_gan/sivasurya_chandran/failure_analysis.md`):

- invented purple colour on a grey, misty field;
- washed-out storm skies;
- speckled texture on close-ups;
- watermarks such as "© 2016 Thomas J. Astle" surviving translation;
- Monet→photo outputs much darker than the painting (Waterloo Bridge becomes a night scene);
- a still life with no photographic equivalent returned almost unchanged.

Reproducibility

- Each member's manifest is in `reproducibility/manifests/` and their raw logs in `reproducibility/raw_logs/<member>/`.
- Smoke test for any of Rajesh's runs: `LAB_SMOKE=1 jupyter nbconvert --to notebook --execute <notebook>`. The README gives the per-task commands.
- Siva's runs are config-driven (`config.yaml` in each of his folders). The exact command for each task is in his `results.md` and in `reproducibility/manifests/sivasurya_chandran.md`.

References

- Vaswani, A., et al. (2017). Attention is all you need. NeurIPS.
- Eldan, R., and Li, Y. (2023). TinyStories: How small can language models be and still speak coherent English? arXiv:2305.07759.
- Zhu, J.-Y., Park, T., Isola, P., and Efros, A. A. (2017). Unpaired image-to-image translation using cycle-consistent adversarial networks. ICCV.
- Zhang, X., Zhao, J., and LeCun, Y. (2015). Character-level convolutional networks for text classification (Yelp polarity). NeurIPS.
- Joulin, A., Grave, E., Bojanowski, P., and Mikolov, T. (2017). Bag of tricks for efficient text classification. EACL.
- Kim, Y. (2014). Convolutional neural networks for sentence classification. EMNLP.
- Bahdanau, D., Cho, K., and Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. ICLR.
- Zhao, S., Liu, Z., Lin, J., Zhu, J.-Y., and Han, S. (2020). Differentiable augmentation for data-efficient GAN training. NeurIPS.
- Miyato, T., Kataoka, T., Koyama, M., and Yoshida, Y. (2018). Spectral normalization for generative adversarial networks. ICLR.
- Heusel, M., et al. (2017). GANs trained by a two time-scale update rule converge to a local Nash equilibrium (FID). NeurIPS.
- Chu, C., Zhmoginov, A., and Sandler, M. (2017). CycleGAN, a master of steganography. arXiv:1712.02950.